Guide pratique TP3 : persistance de données avec JPA et un SGBD relationnel

Remarque : Pour éviter des warnings et des erreurs de l'utilisation de Hibernate/ JPA comme ORM, utiliser de préférence tomcat9 qui supporte le package javax avec Intellij community, et tomcat 11 qui supporte jakarta avec intellij ultimate.

1. Installer le SGBD H2
2. Créer un projet Maven (community) ou Jakarta (ultimate) avec sélection de web application comme archetype.
3. Mise à jour de pom.xml, ajouter les dépendances du tp précèdent par rapport au servlet et JSP

   <u>Ajouter ces dépendances avec intellij ultimate</u>

```
<dependency>
   <groupId>jakarta.servlet.jsp.jstl</groupId>
   <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
   <version>3.0.0</version>
</dependency>
<dependency>
   <groupId>org.glassfish.web</groupId>
   <artifactId>jakarta.servlet.jsp.jstl</artifactId>
   <version>3.0.1</version>
</dependency>

<dependency>
   <groupId>jakarta.ejb</groupId>
   <artifactId>jakarta.ejb-api</artifactId>
   <version>4.0.1</version>
</dependency>

<dependency>
   <groupId>jakarta.persistence</groupId>
   <artifactId>jakarta.persistence-api</artifactId>
   <version>3.2.0</version>
</dependency>

<dependency>
   <groupId>org.hibernate</groupId>
   <artifactId>hibernate-core</artifactId>
   <version>6.0.0.Final</version> <!-- Use a compatible version -->
</dependency>
```

```
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <version>2.2.224</version>
</dependency>
```

Ajouter ces dépendance JSTL avec intellij community

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>

<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version> <!-- Use a compatible version -->
</dependency>

<dependency>
  <groupId>javax.ejb</groupId>
  <artifactId>javax.ejb-api</artifactId>
  <version>3.2.2</version>
</dependency>

<dependency>
  <groupId>javax.persistence</groupId>
  <artifactId>javax.persistence-api</artifactId>
  <version>2.2</version>
</dependency>

<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.2.224</version>
</dependency>
```

4. Création des tables categorie et produit dans H2. Utiliser le script suivant :

```
CREATE TABLE categorie (
   id_categorie INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nom_categorie VARCHAR(100) NOT NULL,
    description VARCHAR(255)
);

CREATE TABLE produit (
    id_produit INT AUTO_INCREMENT PRIMARY KEY,
    nom_produit VARCHAR(100) NOT NULL,
    prix DECIMAL(10,2) NOT NULL,
    quantite INT,
    id_categorie INT,

    CONSTRAINT fk_categorie
    FOREIGN KEY (id_categorie)
    REFERENCES categorie(id_categorie)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

5. Mettre la base en mode server, détecter son URL.
6. Mise à jour du fichier de configuration ressources\META-INF\persistence.xml. Ce fichier permet de localiser la base de données manipulée et de définir les règles d'interaction avec cette base. Penser à utiliser le login et le mot de passe relatif à votre base.
   6.1. persistence.xml version ultimate

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
        version="3.0">
  <persistence-unit name="default">

    <class>com.example.Produit</class>
    <class>com.example.Categorie</class>

  <properties>
```

```
        <!-- H2 -->
        <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
        <property name="jakarta.persistence.jdbc.url"
value="jdbc:h2:tcp://localhost/~/commerce"/>
        <property name="jakarta.persistence.jdbc.user" value="login"/>
        <property name="jakarta.persistence.jdbc.password" value="pswd"/>

        <!-- Hibernate -->
        <property name="hibernate.hbm2ddl.auto" value="update"/>
        <property name="hibernate.show_sql" value="true"/>
        <property name="hibernate.format_sql" value="true"/>
        <property name="jakarta.persistence.lock.timeout" value="5000"/>
      </properties>
      </persistence-unit>
    </persistence>
```

6.2. persistence.xml version community

```
    <?xml version="1.0" encoding="UTF-8"?>
    <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
            xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
            xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
            http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd"
            version="2.2">

      <persistence-unit name="MonPU" transaction-type="RESOURCE_LOCAL">
        <!-- Fournisseur JPA (ex: Hibernate) -->
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <!-- Classes à persister (optionnel si détection auto) -->
        <class>com.exemple.Produit</class>
        <class>com.exemple.Categorie</class>

        <properties>
          <!-- Connexion JDBC -->
          <property name="javax.persistence.jdbc.driver" value="org.h2.Driver"/>
          <property name="javax.persistence.jdbc.url"
value="jdbc:h2:tcp://localhost/~/commerce"/>
          <property name="javax.persistence.jdbc.user" value="sofien"/>
          <property name="javax.persistence.jdbc.password" value="sofien"/>

          <property name="hibernate.hbm2ddl.auto" value="update"/>
          <property name="hibernate.connection.autocommit" value="true"/>
          <!-- Génération du schéma -->

        </properties>
```

```
    </persistence-unit>
  </persistence>
```

7. Création des entités JPA de la couche persistance en sélectionnant Identity comme stratégie de génération de clé primaire pour chaque entité:
   7.1. Utiliser create JPA entities dans la version ultimate. Les entités sont générées automatiquement.
   7.2. Installer à travers settings, plugins le plugin Intellij JPA buddy responsable du reverse Engineering. Les entités doivent être générées pas à pas à travers l'outil Intellij Buddy.

8. Vérifier le code des entités et ajouter en cas de besoin les accesseurs.
9. Création du répertoire java sous main (version community).
10. Ajouter les servlets de votre projet.
11. Développer les JSP nécessaires en utilisant la bibliothèque JSTL.
12. Ajouter le plugin smart tomcat : allez à File, setting, plugins puis chercher, sélectionner et installer le plugin smart tomcat
13. Pour l'exécution allez à Run, edit configurations, puis add new configuration et ajouter smart tomcat.
14. Exécuter et tester l'application
15. Remarque : avec Intellij community, si tomcat n'arrive pas à interpréter les instructions JSTL développées en Expression language, penser à mettre à jour le fichier web.xml de votre projet en utilisant ce script :

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
    version="4.0">
</web-app>
```